

GRID TRADING MASTERY · PART 9

Python Implementation Guide

Project Structure · Core Modules · Bybit API Integration

Backtesting Framework · Paper Trading → Live Migration

การ deploy Bot บน Server + Monitoring



แก่นของ PART นี้

Part นี้ นำ pseudocode จาก Part 1–8 มาประกอบเป็น Python project ที่ runnable ได้จริง
ทุกโค้ดที่เขียนในเล่มนี้ออกแบบมาให้ต่อกันได้ใน architecture ที่แสดงด้านล่าง

9.1 Project Structure

อ่านยังไงถ้าไม่เขียนโค้ด

Part นี้เต็มไปด้วยโค้ด Python — ถ้าไม่ได้ตั้งใจจะ implement เองก็ไม่จำเป็นต้องอ่านทุกบรรทัดให้เน้นอ่าน 3 อย่าง: (1) โครงสร้างไฟล์ด้านล่าง เพื่อเห็นภาพว่าแนวคิดจาก Part 1–8 แต่ละเรื่อง ถูกแบ่งเป็นไฟล์ไหน (2) comment ในโค้ด (บรรทัดที่ขึ้นต้นด้วย #) ซึ่งอธิบายเหตุผล เป็นภาษาคนอ่านง่าย และ (3) กล่อง "ตัวอย่าง"/"สรุป" ท้ายแต่ละหัวข้อ ส่วน syntax เฉพาะของ Python (เช่น @dataclass, class, self) ข้ามได้ถ้าไม่คุ้น — comment ข้างๆ จะบอกว่าโค้ดส่วนนั้นทำอะไรอยู่แล้ว

```
grid-trading-bot/
├── core/
│   ├── __init__.py
│   ├── grid_framework.py      # Zone/Step/Capital/Risk (Part 1A, 3, 5)
│   ├── state_machine.py       # Regime Detection (Part 4)
│   ├── execution_styles.py     # 4 Execution Styles (Part 7B)
│   └── dd_monitor.py           # Drawdown Monitor (Part 5)
├── strategies/
│   ├── buy_only.py             # Part 1A
│   ├── bidirectional.py        # Part 1B
│   ├── hedge_grid.py           # Part 1C
│   ├── pair_grid.py            # Part 6
│   └── snowball.py             # Part 6B
├── indicators/
│   ├── hurst.py                # Hurst Exponent (Part 4)
│   ├── atr_donchian.py         # ATR, Donchian, Keltner (Part 3)
│   ├── composite_score.py      # RSI/BB/Vol/Hurst score (Part 3B)
│   └── kelly.py                # Kelly Criterion (Part 5)
├── exchange/
│   ├── bybit_client.py         # Bybit API wrapper
│   └── data_feed.py            # Market data fetcher
├── backtest/
│   ├── backtest_engine.py      # Event-driven backtester
│   ├── metrics.py              # Sharpe, Calmar, Max DD
│   └── bootstrap_zone.py       # Block Bootstrap (Part 3)
├── monitoring/
│   └── watchdog.py             # Grid Watchdog (Part 4)
```

```
|   ├── alerts.py           # LINE/Telegram notify
|   └── dashboard.py        # Simple web dashboard
└── config/
    ├── config.yaml         # All parameters
    └── secrets.yaml        # API keys (gitignored)
└── tests/
    └── test_*.py           # Unit tests
└── main.py                 # Entry point
└── requirements.txt
```



```

"""
สร้าง GridFramework จากข้อมูลตลาด (Part 3) + คำนวณ grid_capital
จาก
Kelly sizing (Part 5) ในฟังก์ชันนี้เอง – รับ net_worth ดิบเข้ามา
(ไม่ใช่ grid_capital ที่คุณมาแล้ว) เพื่อบังคับให้ kelly_fraction
ถูกใช้จริงทุกครั้งที่เราเรียกฟังก์ชันนี้ ไม่ใช่แค่ parameter ธรรมดา
"""

from indicators.atr_donchian import compute_atr,
compute_donchian

grid_capital = net_worth * kelly_fraction
current_price = prices[-1]

# Zone from Donchian (Part 3)
dc_high, dc_low = compute_donchian(highs, lows, period=30)
zone_high = dc_high * 1.05
zone_low = dc_low * 0.95

# Step from ATR (Part 3) – ปิดทั้งสองขา (ATR-based กับ fee-floor)
เป็น
# หลักร้อยก่อนเทียบ max กัน เพื่อให้ step สุดท้ายเป็นจำนวนเต็มร้อยเสมอ
atr = compute_atr(highs, lows, prices, period=14)
atr_step = round(0.5 * atr / 100) * 100
fee_floor = round(3 * current_price * 0.001 * 2 / 100) * 100
step = max(atr_step, fee_floor)

n = int((zone_high - zone_low) / step)
avg_p = (zone_high + zone_low) / 2
base_q = grid_capital / (n * avg_p)

return cls(symbol=symbol, zone_high=zone_high,
zone_low=zone_low,
step=step, grid_capital=grid_capital,
base_q=round(base_q, 4))

def create_buy_levels(self, current_price, sizes=None, step=None) -
> List[GridLevel]:
    """วาง BUY orders ต่ำกว่า current_price (step=None ใช้ self.step
ปกติ,
หรือรับ step ที่ปรับแล้วจาก execution style เช่น Step Pyramid)"""
    step = step if step is not None else self.step
    levels = []
    price = current_price - step
    i = 0

```

```

while price >= self.zone_low:
    q = sizes[i] if sizes else self.base_q
    levels.append(GridLevel(
        price=round(price, 0),
        qty=q,
        side="BUY",
        tp_price=round(price + step, 0)
    ))
    price -= step
    i += 1
return levels

def allocate_by_weight(self, style, size_mult=1.0) -> List[float]:
    """คำนวณ sizes ตาม allocation style (Part 2)"""
    import math
    n = self.n_levels
    if style == "equal":
        weights = [1.0] * n
    elif style == "bell":
        center = n // 2
        weights = [math.exp(-0.5 * ((i-center)/(n/4))**2) for i in
range(n)]
    elif style == "bottom_heavy":
        weights = [1.0/(n-i) for i in range(n)]
    else:
        weights = [1.0] * n
    total = sum(weights)
    return [self.base_q * (w/total) * n * size_mult for w in
weights]

def create_orders(self, exec_params, current_price) ->
List[GridLevel]:
    """
    แปลง exec_params จาก execution style (Part 7B) เป็น order list
จริง
step/sizes มาจาก exec_params ถ้ามี ไม่จ้้นใช้ default ของ framework
เอง
    """
    if not exec_params.get("deploy", True):
        return []
    return self.create_buy_levels(
        current_price,
        sizes=exec_params.get("sizes"),

```

```
        step=exec_params.get("step")  
    )
```


9.3 Bybit API Integration

ทำไมต้องมี "signature"

Bybit (และ exchange อื่นแทบทุกที่) ต้องพิสูจน์ว่า request ที่ส่งเข้ามาจริงๆ มาจากเจ้าของ API key — ไม่ใช่ใครก็ได้ที่ดักฟัง network แล้วปลอมคำสั่งซื้อขาย วิธีที่ใช้กันคือ HMAC (Hash-based Message Authentication Code): เอา secret key มา "เซ็นชื่อ" ข้อความ (params + timestamp) ให้กลายเป็น hash หนึ่งก้อน แล้วแนบ hash นั้นไปกับ request แทนที่จะส่ง secret key ตรงๆ ฝั่ง server คำนวณ hash แบบเดียวกันด้วย secret key ที่เก็บไว้ แล้วเทียบว่าตรงกันไหม ถ้าตรง = request นี้มาจากเจ้าของ key จริง โค้ดด้านล่าง (`_sign()`) ทำขั้นตอนนี้

```
# exchange/bybit_client.py

import hmac
import hashlib
import time, requests
from typing import Optional

class BybitClient:
    BASE_URL = "https://api.bybit.com"

    def __init__(self, api_key: str, api_secret: str, testnet: bool = True):
        self.api_key = api_key
        self.api_secret = api_secret
        if testnet:
            self.BASE_URL = "https://api-testnet.bybit.com"

    # ⚠ Security: Never log api_secret or the resulting signature.
    # Load api_secret from env var only: os.environ["BYBIT_API_SECRET"]
    def _sign(self, params: dict, timestamp: int) -> str:
        query = "&".join(f"{k}={v}" for k, v in sorted(params.items()))
        msg = f"{timestamp}{self.api_key}5000{query}"
        return hmac.new(self.api_secret.encode(), msg.encode(),
                        hashlib.sha256).hexdigest()

    def _request(self, method: str, endpoint: str, params: dict = None):
```

```

ts = int(time.time() * 1000)
params = params or {}
sig = self._sign(params, ts)
headers = {
    "X-BAPI-API-KEY": self.api_key,
    "X-BAPI-SIGN": sig,
    "X-BAPI-TIMESTAMP": str(ts),
    "X-BAPI-RECV-WINDOW": "5000",
}
url = self.BASE_URL + endpoint
if method == "GET":
    resp = requests.get(url, params=params, headers=headers)
else:
    resp = requests.post(url, json=params, headers=headers)
return resp.json()

def place_order(self, symbol: str, side: str, qty: float, price:
float,
                order_type: str = "Limit") -> dict:
    params = {
        "category": "spot",
        "symbol": symbol,
        "side": side,
        "orderType": order_type,
        "qty": str(qty),
        "price": str(price),
        "timeInForce": "GTC",    # Good Till Cancel
    }
    return self._request("POST", "/v5/order/create", params)

def cancel_all_orders(self, symbol: str) -> dict:
    return self._request("POST", "/v5/order/cancel-all",
                        {"category": "spot", "symbol": symbol})

def get_orderbook(self, symbol: str, limit: int = 5) -> dict:
    return self._request("GET", "/v5/market/orderbook",
                        {"category": "spot", "symbol": symbol,
"limit": limit})

def get_wallet_balance(self, coin: str = "USDT") -> float:
    resp = self._request("GET", "/v5/account/wallet-balance",
                        {"accountType": "UNIFIED"})
    for item in resp.get("result", {}).get("list", [{}])
[0].get("coin", []):

```

```
        if item["coin"] == coin:
            return float(item["walletBalance"])
    return 0.0
```

9.4 Backtesting Engine

```
# backtest/backtest_engine.py

import pandas as pd
import numpy as np
from core.grid_framework import GridFramework, GridLevel

class GridBacktester:
    """
    Event-driven backtester สำหรับ Buy-Only Spot Grid
    """
    def __init__(self, grid: GridFramework, fee_rate: float = 0.001):
        self.grid = grid
        self.fee_rate = fee_rate

    def run(self, ohlcv_df: pd.DataFrame) -> dict:
        """
        ohlcv_df: DataFrame ที่มีคอลัมน์ open, high, low, close, volume
        """
        capital = self.grid.grid_capital
        usdt = capital      # เริ่มต้นด้วย USDT ทั้งหมด
        btc = 0.0           # ไม่มี BTC เริ่มต้น
        inventory = {}      # {buy_price: qty}

        trades = []
        portfolio_values = []

        # สร้าง BUY levels
        current_price = ohlcv_df["close"].iloc[0]
        buy_levels = self.grid.create_buy_levels(current_price)

        for idx, row in ohlcv_df.iterrows():
            low_price = row["low"]
            high_price = row["high"]
            close = row["close"]

            # หมายเหตุ: logic นี้เป็น optimistic assumption
            # ราคา candle แต่ละ level ไม่ได้หมายความว่า fill ทุกครั้ง
            # ใน production ให้ใช้ WebSocket order events แทน
```

```

        # Check BUY fills (price ลงถึง BUY level)
        for level in buy_levels:
            if not level.filled and low_price <= level.price:
                cost = level.qty * level.price * (1 +
self.fee_rate)

                if usdt >= cost:
                    usdt -= cost
                    btc += level.qty
                    inventory[level.price] = level.qty
                    level.filled = True
                    trades.append({"date": idx, "side": "BUY",
                                "price": level.price, "qty":
level.qty})

        # Check SELL fills (TP hit)
        for buy_price, qty in list(inventory.items()):
            tp_price = buy_price + self.grid.step
            if high_price >= tp_price:
                proceeds = qty * tp_price * (1 - self.fee_rate)
                usdt += proceeds
                btc -= qty
                del inventory[buy_price]
                # Reset BUY level
                for level in buy_levels:
                    if level.price == buy_price:
                        level.filled = False

                trades.append({"date": idx, "side": "SELL",
                            "price": tp_price, "qty": qty})

        # Portfolio value
        portfolio_values.append(usdt + btc * close)

    # Metrics
    pv = pd.Series(portfolio_values, index=ohlcv_df.index)
    returns = pv.pct_change().dropna()
    max_dd = (pv / pv.cummax() - 1).min()
    sharpe = returns.mean() / returns.std() * np.sqrt(365) if
returns.std() > 0 else 0

    return {
        "final_portfolio": portfolio_values[-1],
        "total_return": (portfolio_values[-1] / capital - 1),
        "max_drawdown": max_dd,

```

```
    "sharpe_ratio": sharpe,  
    "n_trades": len(trades),  
    "trades": pd.DataFrame(trades),  
    "portfolio_values": pv,  
}
```

9.5 Main Entry Point + Config

```
# config/config.yaml

grid:
  symbol: "BTCUSDT"
  net_worth: 100000      # ใช้ baseline เดียวกับตัวอย่างทั้งเล่ม (Part 1-8) เพื่อให้เทียบตัวเลขกันได้
  kelly_fraction: 0.19  #  $f_{\text{safe}} = 25\% \times f^* \approx 19\%$  (ตาม Part 5 ตัวอย่าง  $f^*=0.754$ ) →  $\text{grid\_capital} \approx \$19,000$ 
  candle_interval: "1d" # เรือนวัด – ทุก indicator ด้านล่างคำนวณจากแท่งนี้ (Part 3 §3.0)
  donchian_period: 30
  atr_period: 14
  step_factor: 0.5      #  $\text{Step} = \text{step\_factor} \times \text{ATR}$ 

risk:
  dd_halt_threshold: -0.08 # -8% drawdown → HALT (Part 5)
  dd_resume_threshold: -0.04 # -4% → resume
  max_notional_per_level: 0.05 # 5% of capital (Part 5)

regime:
  hurst_on: 0.50          # Grid ON ถ้า  $H < 0.50$ 
  hurst_caution: 0.58    # CAUTION ถ้า  $H$  0.50-0.58
  adx_caution: 40        # BTC-calibrated (Part 4): ADX 40-50 → CAUTION
  adx_off: 50             # ADX > 50 → GRID OFF
  bb_width_caution: 0.04
  bb_width_off: 0.08

execution:
  style: "composite"      # mean_reversion / trend_adaptive / volume_adaptive / composite
  recheck_hours: 1        # ตรวจ regime ทุก 1 ชั่วโมง – เรือนตัดสินใจ (Part 3 §3.0)
  tp_policy: "one_step"   # ค่าเดียวที่ GridFramework รองรับตอนนี้ (§9.2) – ทางเลือกอื่นดู Part 2 §2.6.1

snowball:
  enabled: true
  type: 1                 # 1=simple compound, 2=triggered layer, 3=zone
```

```

upgrade
    dca_pct: 0.30          # 30% ของ profit ไป DCA

# === ไฟล์แยก: main.py ===

import yaml
import json
from core.grid_framework import GridFramework
from core.execution_styles import UnifiedGridController
from exchange.bybit_client import BybitClient
from exchange.data_feed import DataFeed
import time, logging

logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s [%(levelname)s] %(message)s")

def main():
    # Load config
    with open("config/config.yaml") as f:
        cfg = yaml.safe_load(f)
    with open("config/secrets.yaml") as f:
        secrets = yaml.safe_load(f)
    # Alternative (recommended): load from env vars instead of
    secrets.yaml
    # api_key = os.environ["BYBIT_API_KEY"]
    # api_secret = os.environ["BYBIT_API_SECRET"]

    # Initialize
    client = BybitClient(secrets["api_key"], secrets["api_secret"],
testnet=True)
    data = DataFeed(client, cfg["grid"]["symbol"])

    # Get market data
    prices, highs, lows, volumes = data.get_ohlcv(days=60)

    # Build framework
    framework = GridFramework.from_market_data(
        symbol=cfg["grid"]["symbol"],
        prices=prices, highs=highs, lows=lows,
        net_worth=cfg["grid"]["net_worth"],
        kelly_fraction=cfg["grid"].get("kelly_fraction", 0.19)
    )
    logging.info(f"Grid: zone=[{framework.zone_low:.0f},
{framework.zone_high:.0f}] ")

```



```

        f"step={framework.step:.0f} Q={framework.base_q:.4f}
N={framework.n_levels}")

    # Controller – run_cycle() ทำครบในเรียกเดียว: DD Halt (Part 5) →
regime
    # (Part 4) → execution style (Part 7B) → order placement. ไม่ต้องมี
    # watchdog แยกมาห่อหุ้มอีกชั้น เพราะ watchdog ที่แยกออกไปมักลิมเช็ค DD Halt
    # และ execution style ครบทั้งคู่ – รวมไว้ในฟังก์ชันเดียวปลอดภัยกว่า
    controller = UnifiedGridController(framework,
style=cfg["execution"]["style"])

    # Main loop
    portfolio_history = [framework.grid_capital]

    while True:
        # ต้อง refresh ข้อมูลตลาดทุกรอบ ไม่ใช่ prices ก่อนเดิมจากตอน start
        prices, highs, lows, volumes = data.get_ohlcw(days=60)
        market_data = data.get_indicators(prices, highs, lows, volumes)
        market_data["current_price"] = prices[-1]
        market_data["portfolio_history"] = portfolio_history

        result = controller.run_cycle(market_data)
        logging.info(f"Action: {result['action']}")

        if result["action"] == "HALT":
            client.cancel_all_orders(cfg["grid"]["symbol"])
            logging.warning(f"HALTED: {result['reason']}")

        elif result["action"] == "PAUSE":
            logging.info(f"Paused (state={result['state']}) – ไม่เปิด
order ใหม่รอบนี้")

        elif result["action"] == "EXECUTE":
            for order in result["orders"]:
                resp = client.place_order(
                    symbol=cfg["grid"]["symbol"],
                    side=order.side,
                    qty=order.qty,
                    price=order.price
                )
                logging.info(f"Order placed: {resp}")

        # Update portfolio value
        balance = client.get_wallet_balance("USDT")

```

```

portfolio_history.append(balance)

# เขียน state ล่าสุดให้ dashboard (§9.8) อ่านได้
with open("state.json", "w") as f:
    json.dump({"action": result["action"], "portfolio_value":
balance,
                "n_levels": framework.n_levels}, f)

# ⚠ Production Note: sleep(3600) จะพลาด order fills ระหว่างรอ
# ใช้ WebSocket private channel
(wss://stream.bybit.com/v5/private) แทน polling
# ดู Bybit API docs: subscribe to "order" channel เพื่อรับ fill
events real-time
# ⚠ Production: implement exponential back-off reconnect –
connection drop = missed fills
# while True:
#     try:
#         ws.run_forever()
#     except ConnectionError:
#         time.sleep(min(2 ** attempt, 60)) # รอเพิ่มขึ้นเรื่อยๆ
สูงสุด 60s
#         attempt += 1
#         time.sleep(cfg["execution"]["recheck_hours"] * 3600)

if __name__ == "__main__":
    main()

```

9.6 Paper Trading → Live Migration

Phase	ระยะ เวลา	ทำอะไร	Success Criteria
Phase 0: Backtest	1 สัปดาห์	Run backtester บน 1–2 ปีย้อนหลัง	Sharpe > 1.5, Max DD < 15%
Phase 1: Paper Trade	2–4 สัปดาห์	Run bot บน Testnet (สภาพแวดล้อมทดสอบของ Bybit — API จริง แต่เงินจำลอง ตั้ง testnet=True)	ไม่มี bug, state machine ทำงาน
Phase 2: Small Live	1 เดือน	\$1,000 จริง บน Bybit	P&L $\approx$ backtest, DD $\geq$ -8% (ไม่ชน HALT)
Phase 3: Scale Up	ต่อเนื่อง	เพิ่ม capital หลังผ่าน Phase 2	Compound + DCA Stack

9.7 Pre-Live Checklist — สิ่งที่ต้องตรวจก่อน Live

ตรวจทุกข้อก่อน switch testnet=False

- API Rate Limit: Bybit อนุญาต 600 requests/นาที → grid 30 levels × 2 (place/cancel) = 60/cycle ✓
- Minimum Order Size: BTC บน Bybit ขั้นต่ำ 0.001 BTC → Q ต้องมากกว่านี้
- Price Precision: BTC round ถึง \$0.01 → step ต้องเป็น multiple ของ \$0.01
- Network Redundancy: รัน bot บน VPS (Virtual Private Server — เซิร์ฟเวอร์เช่าที่ online ตลอด 24/7) พร้อม reconnect logic ไม่ควรรัน bot บน local machine ที่อาจปิดหรือ internet หลุด
- Secret Management: API key เก็บใน environment variable (ค่าที่ตั้งไว้นอกโค้ด อ่านตอนรัน โปรแกรม) ไม่ใช่ hardcode (พิมพ์ค่าจริงติดอยู่ในซอร์สโค้ดตรงๆ) เพราะไฟล์โค้ดอาจหลุดเข้า git repo หรือถูกแฮกโดยไม่ตั้งใจ

9.8 Monitoring Dashboard

Flask คืออะไร

Flask เป็น web framework เล็กๆ ของ Python — ใช้เปิดหน้าเว็บง่ายๆ ที่รับ request แล้วตอบกลับเป็น HTML หรือ JSON ในที่นี้ใช้ทำ dashboard ดู status ของ bot ผ่านเบราว์เซอร์: โค้ดด้านล่างอ่าน state.json (ไฟล์ที่ main loop ใน §9.5 เขียนสถานะล่าสุดไว้ทุกรอบ หลังจากอัปเดต portfolio value) แล้วส่งกลับเป็น JSON ให้หน้าเว็บแสดงผล ส่วน HTML/JS ที่ฝังไว้ใน string เป็นแค่หน้าแสดงผลง่ายๆ ฝัง browser ไม่เกี่ยวกับ logic การเทรดใดๆ

```
# monitoring/dashboard.py (Flask simple dashboard)

from flask import Flask, jsonify, render_template_string
import json

app = Flask(__name__)

HTML = """
<!DOCTYPE html><html><head><title>Grid Dashboard</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script></head>
<body style="font-
family:monospace;padding:20px;background:#0f172a;color:#e2e8f0">
<h2>Grid Bot Dashboard</h2>
<div id="status">Loading...</div>
<canvas id="chart" width="800" height="300"></canvas>
<script>
fetch('/api/status').then(r=>r.json()).then(data=>{
  document.getElementById('status').innerHTML =
    '<pre>' + JSON.stringify(data, null, 2) + '</pre>';
});
</script>
</body></html>
"""

@app.route("/")
def dashboard():
    return render_template_string(HTML)
```

```

@app.route("/api/status")
def status():
    # อ่านจาก state file ที่ bot เขียนไว้
    try:
        with open("state.json") as f:
            return jsonify(json.load(f))
    except FileNotFoundError:
        return jsonify({"error": "Bot not running"})

# รัน: python -m monitoring.dashboard
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)

```

สรุปเนื้อหาทั้งเล่ม — Cheat Sheet

GRID TRADING MASTERY – Quick Reference

TIMEFRAME: แท่งรายวัน (1D) เป็น default ทุก indicator ด้านล่าง (Part 3 §3.0)

เปลี่ยนแท่งต้อง derive สูตรใหม่เอง – ห้ามเอาตัวเลขจาก daily ไปใช้
ตรงๆ

TP: 1-step (default, ตายตัวทุกสูตรในเล่มนี้) | ทางเลือกอื่นดู Part 2 §2.6.1

ZONE: Donchian(30) $\pm$ 5% buffer | min_width $\geq$ 6 $\times$ ATR

STEP: max(0.5 $\times$ ATR(14), 3 $\times$ fee_per_cycle) $\rightarrow$ round to \$100

Q: grid_capital / (N $\times$ avg_price) $\rightarrow$ ตรวจ 5% max notional

N: zone_width / step $\rightarrow$ ควรอยู่ระหว่าง 15-25

KELLY: $f^* = (p \times b - q) / b$ | $f_{\text{safe}} = 0.25 \times f^*$

grid_capital = net_worth $\times$ f_{safe}

REGIME (Part 4, BTC-calibrated):

H < 0.50 + ADX < 40 + BB_width < 4% $\rightarrow$ GRID ON (full deploy)

H 0.50-0.58 + ADX 40-50 + BB_width 4-8% $\rightarrow$ CAUTION (50% size)

H > 0.58 + ADX > 50 + BB_width > 8% $\rightarrow$ GRID OFF (pause)

RISK (Part 5):

DD $\leq$ -8% $\rightarrow$ HALT (cancel all, wait for recovery)

DD $\geq$ -4% + H < 0.55 + 24h cooloff $\rightarrow$ RESUME

SNOWBALL (Part 6B):

Type 1: $Q(t) = Q_0 \times (1 + r_{\text{daily}})^t$
DCA Stack: 30% profit → weekly BTC buy

COMPOSITE SCORE (Part 3B):

$30\% \times \text{RSI} + 30\% \times \text{BB} + 20\% \times \text{Vol} + 20\% \times \text{Hurst} \rightarrow 0-100$

>75: Full Deploy | 55-75: 60% | 35-55: 30% | <35: Wait

แบบฝึกหัด Part 9

- ▶ ข้อ 1: อ่าน config.yaml แล้วอธิบายว่า `kelly_fraction=0.19` หมายความว่าอย่างไร และ `net_worth=$100,000` ใน config ทำให้ `GridFramework.from_market_data()` คำนวณ `grid_capital` ได้เท่าไร?
- ▶ ข้อ 2: ใน `run_cycle()` ถ้า `DD = -9%` เกิดอะไรขึ้น? bot ยังคงทำงานไหม?
- ▶ ข้อ 3: ทำไม `testnet=True` ต้องถูกเปลี่ยนเป็น `False` ก่อน Live? มีความเสี่ยงอะไรถ้าลืม?
- ▶ ข้อ 4: Backtester ใน Part 9 มี "optimistic assumption" อะไรที่ทำให้ผลดีกว่าความเป็นจริง?